

ResearchPilot: An Agent For Systematic Literature Review

Tisha Bhavsar

Plaksha University/ Advanced NLP

tisha.bhavsar.ug23@plaksha.edu.in

<https://github.com/tishakbhavsar/ResearchPilot.git>

1 Problem Statement

Conducting a literature review is one of the most time-consuming tasks in academic research. A researcher must translate a vague topic into effective search queries, retrieve candidate papers from multiple sources, rank them by relevance, read and critically analyze each one, and synthesize findings across papers into a coherent narrative. Each sub-task requires a different kind of reasoning—keyword extraction, retrieval, critical reading, and cross-paper comparison—which makes it a natural fit for a multi-step LLM agent (Yao et al., 2022).

A single prompt cannot accomplish this reliably. A prompt asking for a literature review on a topic produces hallucinated citations and shallow summaries drawn from the model’s parametric memory — it cannot retrieve real papers, accurately report citation counts, or compare methodologies across works it has not been shown. Multi-step chaining is necessary because each of these sub-tasks depends on the output of the one before it: structured search terms must exist before any retrieval can happen, retrieved and scored papers must exist before the best candidates can be selected, individual paper analyses must exist before cross-paper patterns can be identified, and a complete synthesis must exist before a coherent report can be assembled. No stage can be skipped or reordered without breaking what follows. The result is a grounded, traceable research brief that a researcher could act on directly (Lewis et al., 2020; Bubeck et al., 2023).

2 Chain Design

ResearchPilot implements a six-step sequential pipeline managed through a shared Python state dictionary that accumulates outputs across the chain. Each step depends on artifacts produced by earlier stages, making the pipeline strictly ordered.

- **Step 1: Query Structuring.** Receives

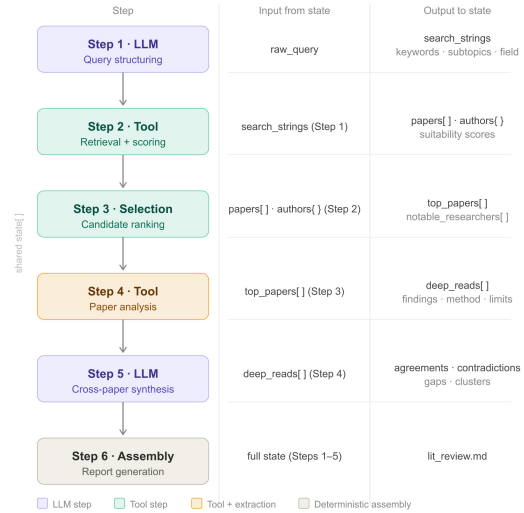


Figure 1: ResearchPilot multi-step literature review pipeline.

state["raw_query"], a plain-English research topic. The LLM converts it into a structured JSON object containing the canonical topic, field, subtopics, keywords, time range, and optimized search queries. This step grounds downstream retrieval and subtopic coverage analysis.

- **Step 2: Retrieval and Scoring.** Receives state["search_strings"] and queries the Semantic Scholar Academic Graph API for candidate papers. Papers are ranked using a deterministic relevance score combining keyword overlap, citation count, recency, and open-access availability. The resulting paper and author metadata are written to state["papers"] and state["authors"]. This stage is tool-driven because citation statistics and paper metadata cannot be reliably generated by the LLM.
- **Step 3: Candidate Selection.** Receives the scored paper list and author profiles. A

deterministic ranking stage selects the top-10 papers and assembles researcher profiles using h-index metadata retrieved earlier.

- **Step 4: Paper Analysis.** Receives state["top_papers"]. Using abstracts retrieved in Step 2, a deterministic extraction layer identifies the core argument, method category, key findings, and limitations for each paper. This replaced earlier per-paper LLM calls after testing showed significant rate-limit and latency issues on free-tier APIs.
- **Step 5: Cross-Paper Synthesis.** Receives all paper analyses simultaneously. A deterministic synthesis stage identifies recurring themes, agreements, contradictions, and research gaps using frequency and polarity heuristics. An optional lightweight LLM refinement step improves thematic labels while remaining within token limits.
- **Step 6: Report Assembly.** Receives the fully accumulated state and compiles the final Markdown research brief, including ranked papers, researcher profiles, per-paper summaries, thematic clusters, contradictions, and open research gaps. The final implementation uses deterministic report assembly to avoid truncation and verbosity issues observed with large generative outputs.

3 Tool Integration

The primary external tool is the *Semantic Scholar Academic Graph API* (Ammar et al., 2018), used in Step 2 for academic paper retrieval. It was selected because it provides structured metadata including titles, abstracts, citation counts, author identifiers, external IDs, open-access links, and author-level metrics such as h-index within a single free API ecosystem.

Retrieved metadata propagates through scoring, ranking, paper analysis, and final report generation stages, making the retrieval layer the grounding mechanism for the entire pipeline.

A secondary fallback uses the *OpenAlex* (Priem et al., 2022) API when Semantic Scholar returns insufficient results. To improve reliability under free-tier constraints, the system implements exponential backoff retries and an on-disk cache (.s2_cache/) that stores API responses between runs, reducing repeated requests during development and demonstrations.

4 Limitations

The current system performs abstract-only analysis rather than full-paper parsing. Earlier versions retrieved and processed full PDFs, producing substantially deeper literature reviews with richer methodological comparisons and longer synthesis outputs. However, full-text processing significantly increased runtime, with end-to-end execution frequently exceeding 10 minutes during testing and demos. To maintain acceptable response time, the final pipeline was simplified to abstract-level analysis.

As a result, detailed experimental settings, numerical results, and deeper methodological nuances are often unavailable to the synthesis stage. This weakens contradiction detection and limits the depth of cross-paper comparisons that the system can perform reliably.

The pipeline is also constrained by external API limitations. Semantic Scholar rate limiting affected retrieval reliability during development, requiring API access requests and retry handling mechanisms. Similarly, free-tier LLM APIs imposed token and request constraints that influenced the final system design.

5 Reflection

One of the main engineering challenges was balancing output quality against runtime and API constraints. Early versions of the system used larger generative stages and full-paper parsing, producing more detailed literature reviews but causing long execution times, token overflows, and truncated JSON outputs. After experimentation with multiple LLM providers, Gemini was adopted as a substitute for Grok, though token-limit issues still required simplifying later stages into deterministic processing steps.

Designing the chaining pipeline itself was one of the most valuable parts of the project. Considerable effort went into determining which stages should remain LLM-driven and which could be replaced with deterministic logic for reliability and transparency. Building a literature review agent also required careful thinking about how papers should be ranked and scored in a traceable way rather than relying entirely on opaque LLM outputs (Lewis et al., 2020).

The project provided practical insight into how multi-step AI agents are engineered under real-world constraints such as rate limits, latency, re-

trieval reliability, and context-window limitations.

References

- Waleed Ammar and 1 others. 2018. Construction of the literature graph in semantic scholar. In *NAACL*.
- Sebastien Bubeck and 1 others. 2023. Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv:2303.12712*.
- Patrick Lewis and 1 others. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*.
- Jason Priem and 1 others. 2022. Openalex: A fully-open index of scholarly works, authors, venues, institutions, and concepts. *arXiv preprint arXiv:2205.01833*.
- Shunyu Yao and 1 others. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.

APPENDIX

A Prompt Design Explanation

This appendix documents the prompts used in each LLM stage of the pipeline, the reasoning behind their structure, and the changes made after testing. The prompts were designed so that every stage produced structured outputs that could be consumed reliably by downstream steps.

A.1 Step 1 — Query Structuring

Purpose. The first LLM call converts a vague user topic into structured fields required by the retrieval pipeline: keywords, subtopics, time range, and search queries. Step 2 depends directly on these fields for Semantic Scholar retrieval.

Full System Prompt

You are a research librarian and
→ academic search specialist.
Your job is to parse a user's research
→ topic into a structured JSON object
→ that
will drive an automated literature
→ review pipeline.

Rules:

- Return ONLY a valid JSON object. No
→ prose, no markdown fences, no
→ explanation.
- All string values must be in English.
- search_strings must be short enough to
→ use directly as Semantic Scholar API
queries (36 words each, no boolean
→ operators, no quotes, no AND/OR).
- keywords must be single words or short
→ noun phrases (no full sentences).
- time_range should be a string like
→ "20182024". Pick based on field
→ velocity:
fast-moving (deep learning, NLP) last
→ 56 years.
slower (theory, linguistics) last 810
→ years.

Output schema return exactly this

→ structure:
{
 "topic": "<canonical one-line topic
 → title>",
 "field": "<primary academic
 → discipline>",

```
"time_range": "<e.g. 2019-2024>",  
"keywords": ["kw1", "kw2", "kw3"],  
"subtopics": ["subtopic1",  
  → "subtopic2"],  
"search_strings": [  
  "<query string 1>",  
  "<query string 2>",  
  "<query string 3>"  
]  
}
```

Full User Prompt

Parse the following research topic for a
→ literature review pipeline.

Research topic: "{raw_query}"

Return the JSON object only.

Design Rationale The prompt enforces strict JSON output because downstream parsing uses `json.loads()`. Search strings were constrained to short keyword phrases because long boolean-style queries performed poorly on the Semantic Scholar API. The time-range heuristic prevents the model from returning excessively broad ranges such as "all years."

Prompt Iteration An earlier version frequently produced Markdown-fenced JSON. This caused parser failures. The final prompt explicitly added: "*Return ONLY a valid JSON object. No prose, no markdown fences.*" which significantly improved reliability.

A.2 Step 3 — Candidate Selection

Purpose. Step 3 selects the most relevant papers from the retrieved candidate pool. The numeric scoring formula in Step 2 cannot determine whether papers collectively cover all subtopics or whether a highly cited paper is semantically relevant.

Full System Prompt

You are a researcher selecting 5 key
→ papers from a candidate list.

Rules:

Return ONLY valid JSON. No prose.
top_paper_ids: list of exactly 5 IDs
→ copied from candidates.
selection_reasoning: 1-2 sentences only.
notable_researchers: optional short list
→ of 3 names.

Schema:

```
{
  "top_paper_ids": ["id1", "id2", "id3",
    ↪ "id4", "id5"],
  "selection_reasoning": "reason",
  "notable_researchers": []
}
```

Full User Prompt

Topic: {topic}
Subtopics: {subtopics}

--- PAPERS (pick 5 by ID) ---
{papers_block}

--- AUTHORS ---
{authors_block}

Return JSON.

Design Rationale The output uses paper IDs instead of titles because title variations caused lookup failures during testing. The prompt explicitly enforces “exactly 5” papers to prevent inconsistent output sizes. Researcher extraction was also placed in this stage because author h-index metadata is already available here.

Prompt Iteration An earlier version requested paper titles instead of IDs. Minor title mismatches between the LLM output and Semantic Scholar metadata caused Step 4 lookups to fail, resulting in empty paper lists. The prompt was redesigned to require exact paper IDs copied from the candidate set.

A.3 Step 4 — Paper Analysis

Purpose. This stage extracts structured signals from each paper abstract individually. The resulting summaries feed directly into cross-paper synthesis in Step 5.

Full System Prompt

You are extracting structured signals
↪ from a research paper abstract.

Return ONLY valid JSON.

```
{
  "core_argument": "<ONE sentence>",
  "method_tag": "<3-5 word method label>",
  "key_findings": ["finding 1", "finding
    ↪ 2"],
```

```
"limitations": ["one limitation"]
}
```

Rules:

core_argument must be exactly ONE
↪ sentence.
method_tag must be 3-5 words.
key_findings must have exactly 2 items.
limitations must have exactly 1 item.

Full User Prompt

Topic: "{topic}"

Paper: {title} ({year})

Abstract:
{paper_text}

Return the 4-field JSON now.

Design Rationale The schema intentionally limits output size to reduce truncation risk. The method_tag field was introduced so Step 6 could group papers by methodology rather than free-text summaries. Findings and limitations were capped because abstracts rarely contain detailed experimental discussion.

Prompt Iteration Earlier versions requested paragraph-length methodology descriptions, multiple findings, and citation extraction. These prompts frequently exceeded token limits and produced truncated JSON outputs. The final version reduced the schema to four short fields, resulting in significantly more reliable generation.

A.4 Step 5 — Cross-Paper Synthesis

Purpose. This is the first stage with visibility across all analyzed papers simultaneously. It identifies agreements, contradictions, research gaps, and thematic clusters.

Full System Prompt

Synthesise 5 papers into JSON with:
agreements, contradictions, gaps,
↪ clusters.

Return ONLY valid JSON.

Schema:
{
 "agreements": [...],

```
"contradictions": [...],
"gaps": [...],
"clusters": [...]
}
```

Full User Prompt.

Topic: {topic}

```
--- ALL 5 PAPERS ---
{deep_reads_block}
```

Return JSON synthesis.

Design Rationale The output format separates agreements, contradictions, gaps, and clusters into explicit arrays so that Step 6 can assemble the final report deterministically. Cluster labels were constrained to short thematic phrases because they become section headers in the final literature review.

Prompt Iteration The initial version returned free-form prose rather than structured JSON. This made it difficult for Step 6 to distinguish contradictions from agreements programmatically. The final version enforced explicit JSON arrays and required contradictions to reference specific papers directly.

A.5 Step 6 — Research Gaps and Navigation

Purpose. Most of Step 6 is deterministic report assembly. The final LLM call is restricted to generating research gaps and a navigation guide for readers.

Full System Prompt

You are a research navigator.

Return ONLY valid JSON with:

```
{
  "gaps": ["gap a", "gap b"],
  "navigation": [
    {"subtopic": "subtopic name",
     ↪ "paper_numbers": [1,3]}
  ]
}
```

Rules:

gaps must be open research questions.
navigation must use provided clusters.
Return ONLY JSON.

Full User Prompt

```
Topic: {topic}
User focus: {user_note}
Themes/clusters: {clusters}
```

```
Papers:
{titles_block}
```

Return the gaps + navigation JSON now.

Design Rationale Research gaps require reasoning about what is missing across all papers, which could not be computed deterministically. The navigation guide helps readers identify which papers are most relevant for each subtopic.

Prompt Iteration An earlier version asked the LLM to generate the entire literature review in one call. This consistently caused token truncation, long runtimes, and overly verbose outputs. The final implementation moved report assembly into deterministic code and reduced the LLM role to only the parts requiring semantic synthesis.

A.6 Example Pipeline Execution

The following example shows a condensed execution trace for the query: *"hallucination detection in LLMs"*.

STEP 1 – Query Parsing

Input:

```
{"raw_query": "hallucination detection
↪ in LLMs"}
```

Output:

```
{
  "topic": "Hallucination Detection in
↪ Large Language Models",
  "keywords": ["hallucination",
↪ "factuality", "reliability"],
  "search_strings_count": 3
}
```

STEP 2 – Retrieval and Ranking

Output:

```
{
  "papers_fetched": 30,
  "authors_profiled": 8
}
```

STEP 3 – Candidate Selection

Output:

```
{
  "top_papers_selected": 10,
  "researchers_profiled": 3
}
```

```
}
```

STEP 4 – Abstract Analysis

Output:

```
{  
  "papers_read": 10  
}
```

STEP 5 – Cross-Paper Synthesis

Output:

```
{  
  "agreements_extracted": 2,  
  "contradictions_found": 1,  
  "gaps_identified": 3  
}
```

STEP 6 – Final Report Generation

Output:

```
{  
  "report_words": 1146  
}
```